
1. SYSTEM OVERVIEW (MANDATORY)

Architecture (high-level)

- **Frontend**
 - Web: React (Next.js preferred)
 - Mobile: React Native
- **Backend**
 - Supabase (DB + Storage)
 - Serverless APIs (Node / Edge functions)
- **Authentication**
 - Better Auth (Primary Authentication System)
- **AI Layer**
 - LLM (OpenAI / similar)
 - RAG (Vector DB – Supabase pgvector / Pinecone)
- **Data Sources**
 - Scraped university data
 - Admin-managed data

2. AI ADVISOR CHAT (CORE PRODUCT)

Goal

Conversational assistant for:

- discovery
- guidance
- conversion push

Key Capabilities

1. Intent Detection

System must classify:

- recommendation
 - comparison
 - eligibility
 - timeline
 - visa
 - generic
-

2. Entity Extraction

Extract from query:

- country
 - course
 - budget
 - GPA
 - intake
-

3. Conversation Flow

Example Flow:

User: *"I want to study abroad"*

System:

1. Detect vague intent
 2. Ask:
 - Preferred country?
 - Budget?
 - Course?
 3. Store responses in session
 4. Trigger UniMatch when enough data
-

4. RAG Integration

Sources:

- University DB
- Visa rules
- FAQs

Flow:

1. Query vector DB
2. Retrieve top 5 docs
3. Pass into LLM prompt

5. Prompt Template (IMPORTANT)

You are an expert study abroad advisor.

User profile:

- GPA: {{gpa}}
- Budget: {{budget}}
- Country: {{country}}

Context:

{{retrieved_docs}}

User query:

{{query}}

Instructions:

- Be concise
- Avoid hallucination
- If unsure, ask clarification

6. Fallback Logic

Scenario	Action
No RAG data	General answer
LLM fails	Static fallback
Ambiguous	Ask question

7. Memory

- Store:
 - country
 - budget
 - course
 - Session-based (no PII stored long-term unless consent)
-

3. LEAD CAPTURE (CONVERSION ENGINE)

Flow

Step-based Form

1. Personal Info
2. Academic Info
3. Preferences
4. Budget
5. Submit

Fields

- Name
- Email
- Phone
- GPA
- IELTS
- Preferred country
- Budget

Logic

Validation

- Email format
- GPA range
- Required fields

Progressive Save

- Save after each step
- Resume later

Lead Scoring

Factor	Score
Full profile	+50
IELTS available	+20
Budget defined	+20

4. UNIMATCH ENGINE (MOST IMPORTANT LOGIC)

Input

```
{  
  "gpa": 7.5,  
  "ielts": 7,  
  "budget": 2500000,  
  "country": "Australia"  
}
```

Processing Steps

1. Normalize Data

- Convert GPA → %
 - Normalize budget currency
-

2. Filter

- Country match
 - Budget range
 - IELTS cutoff
-

3. Scoring Algorithm

Score =
(0.4 * GPA fit) +
(0.3 * IELTS fit) +
(0.2 * Budget fit) +
(0.1 * Ranking weight)

4. Ranking

- Sort descending
 - Remove duplicates
-

5. Output

- University list
 - Match %
 - Reason
-

6. Edge Cases

Case	Handling
No results	Relax filters
Low GPA	Show safe universities
Low budget	Suggest alternate countries

5. RESUME / SOP GENERATOR

Inputs

- Academic background
 - Work experience
 - Goals
 - Country
-

SOP Generation Flow

1. Validate input
 2. Generate sections:
 - Intro
 - Background
 - Goals
 - Why this country
 3. Return editable text
-

Prompt

Write a Statement of Purpose for a student applying to {{country}}.

Details:

{{user_input}}

Tone: Professional, persuasive

Length: 600-800 words

Features

- Edit
 - Regenerate
 - Download (PDF)
-

6. ADMIN DASHBOARD

Modules

1. Leads

- List
- Filter (country, score)
- Status (new / contacted / converted)

2. Lead Detail

- Full profile
- Notes
- Activity history

3. Actions

- Update status
 - Add notes
-

7. COURSE MANAGEMENT ENGINE

Scraper

- Scrape:
 - universities
 - courses
 - fees
 - IELTS requirements

Processing

- Normalize fields
- Remove duplicates
- Store structured data

Admin Panel

- Edit data
- Approve / reject
- Track last updated

8. ANALYTICS TRACKING (VERY IMPORTANT)

Events to Track

- Search
- Recommendation view
- Click
- Lead submit

Event Schema

```
{  
  "event": "search",  
  "query": "MBA Canada",  
  "timestamp": "...",  
  "user_id_hash": "..."  
}
```

Rules

- No PII storage
 - Deduplicate events
-

9. ANALYTICS DASHBOARD

Metrics

- Top searches
- Popular universities
- Conversion rate

Views

- Last 7 / 30 days
 - Filters
-

10. PROVIDER DASHBOARD (NEW)

Users

- Universities / partners
-

Features

1. Applications

- List of applicants
 - Filters
-

2. Document Access

- Download SOP / resume
-

3. Status Update

- Applied → Offer → Rejected
-

4. Offer Upload

- Upload offer letter (PDF)

NON-FUNCTIONAL REQUIREMENTS (DON'T SKIP)

Performance

- Chat response < 3 sec
- Dashboard load < 2 sec

Security

- No raw PII storage
 - Hash sensitive fields
-

AUTH & TECH STACK SPEC

Tech Stack Overview

Frontend

- Web: React (Next.js preferred)
 - Mobile: React Native
-

Backend / Database

- Supabase (PostgreSQL)
 - Used for:
 - User data
 - Leads
 - Universities
 - Applications
 - Analytics events
-

Authentication Layer

- **Better Auth (Primary Authentication System)**
-

Authentication Design

Login Method

- Email-based **Magic Link (Passwordless) ONLY**
-

Why this approach

- No password management
 - Simpler UX
 - Easier migration to future systems (Convex)
 - Lower security overhead
-

User Model (MANDATORY)

System **MUST** maintain its own user table (NOT dependent on Better Auth or Supabase)

```
{
  "id": "internal_user_id (UUID)",
  "email": "user@email.com",
  "auth_provider": "better-auth",
  "provider_user_id": "better_auth_user_id",
  "role": "student | admin | provider",
  "created_at": "",
  "last_login": ""
}
```

Critical Rule (VERY IMPORTANT)

DO NOT use Better Auth user ID directly anywhere in business logic

All systems must use:

internal_user_id

Authentication Flow

Signup / Login

1. User enters email
 2. Better Auth sends magic link
 3. User clicks link → authenticated
 4. Backend receives session
 5. Backend:
 - extracts provider_user_id
 - checks if user exists
 - if not → create new user
 6. Map to internal_user_id
-

Request Flow (ALL APIs)

Every API must:

1. Validate Better Auth session
2. Extract user identity

3. Map to internal_user_id
4. Apply role-based authorization

User Roles

Role	Access
Student	Chat, UniMatch, SOP, Lead
Admin	Dashboard, analytics, data
Provider	Applications, status updates

Authorization Rules

- Role-based access control (RBAC)
- Enforced at API level (NOT frontend)

Migration Strategy (Future: Convex)

System must be designed so that:

- Auth provider can be switched without breaking system
 - Only auth_provider and provider_user_id change
-

Migration Steps (Future)

1. Replace Better Auth or keep it (recommended: keep)
 2. Replace Supabase DB → Convex
 3. Maintain same internal_user_id
-

Session Handling

- Use Better Auth session tokens
 - Frontend stores session securely
 - Backend validates on every request
-

Security Requirements

- No password storage
 - No raw sensitive data in logs
 - Email is primary identity
 - Hash any sensitive identifiers if needed
-